

Docker Containerization - Complete Guide for Time-Aware RAG Project

Phase 4, Part 1: Docker Containerization

WHAT IS DOCKER? (Beginner Friendly)

The Problem Docker Solves

Imagine you built this RAG project on your laptop. Now your colleague wants to run it. They need to:

1. Install Python 3.11 (they have 3.9 — things break)
2. Install PostgreSQL (wrong version — things break)
3. Install Redis (never heard of it — confused)
4. Create database, tables, indexes manually
5. Install all pip packages (some fail on Windows/Linux)
6. Configure .env file correctly
7. Debug for 2 hours... still doesn't work

The Docker Solution

Your colleague does ONE thing:

bash

```
docker-compose up --build
```

Everything works. On any computer. Every single time.

Think of Docker Like This:

1. Docker Container = A mini computer inside your computer
It has its own Linux OS, its own Python, its own everything
It runs your app in an isolated, reproducible environment
2. Dockerfile = Recipe card
"Take Python 3.11, copy my code, install packages, run server"
Builds ONE container
3. docker-compose = Full meal plan
"I need 3 things together: my app + PostgreSQL + Redis"
Orchestrates MULTIPLE containers together
4. .dockerignore = Packing list of what NOT to include
"Don't copy venv/, __pycache__/, .env into the container"
5. .env.docker = Settings specifically for Docker environment
Uses container names instead of localhost

BEFORE vs AFTER DOCKER (Visual)

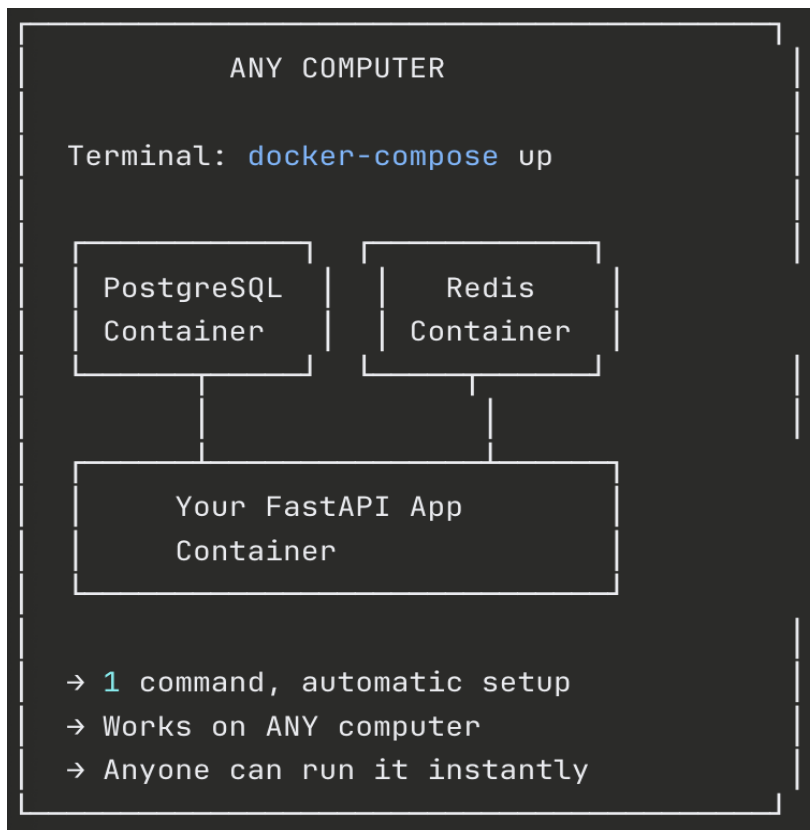
BEFORE DOCKER (manual setup):

```
YOUR LAPTOP

Terminal 1: brew services start
            postgresql
Terminal 2: redis-server
Terminal 3: source venv/activate
            uvicorn app.main:app

→ 3 terminals, manual setup
→ Works ONLY on your machine
→ Colleague can't run it easily
→ "It works on my machine" problem
```

AFTER DOCKER (containerized):



PREREQUISITES

1. Install Docker Desktop

Mac:

- Go to <https://www.docker.com/products/docker-desktop/>
- Download Docker Desktop for Mac
- Install and open it
- Skip the sign-in (click "Skip")
- Wait for the whale icon  to appear in your top menu bar

Windows:

- Go to <https://www.docker.com/products/docker-desktop/>
- Download Docker Desktop for Windows
- Install (enable WSL2 if prompted)

- Restart your computer
- Open Docker Desktop, skip sign-in

Linux:

bash

```
sudo apt-get update
sudo apt-get install docker.io docker-compose-v2
sudo systemctl start docker
sudo usermod -aG docker $USER
# Log out and log back in
```

2. Verify Installation

Open Terminal and run:

bash

```
docker --version
```

Expected output (version may vary):

Docker version 28.x.x, build xxxxxxx

Then run:

bash

```
docker-compose --version
```

Expected output:

Docker Compose version v2.x.x

If both show version numbers, you're ready!

3. Make Sure Docker Desktop is Running

You MUST have Docker Desktop open and running before using any docker commands.

Look for the whale icon  in your menu bar (Mac) or system tray (Windows).

📌 PROJECT STRUCTURE (What We'll Create)

Starting point — your backend folder should look like this:

```
backend/
├── app/
│   ├── __init__.py
│   ├── api/
│   │   ├── __init__.py
│   │   └── routes.py
│   ├── config.py
│   ├── main.py
│   └── models.py
├── core/
│   ├── __init__.py
│   ├── cache.py
│   ├── database.py
│   ├── embeddings.py
│   ├── metadata_store.py
│   ├── pdf_processor.py
│   ├── retriever.py
│   └── vector_store.py
├── llm/
│   ├── __init__.py
│   └── gemini_client.py
├── data/
├── venv/
├── .env
├── .gitignore
└── requirements.txt
```

After Docker setup, we'll add these NEW files:

```
backend/
├── ... (all existing files)
├── Dockerfile          ← NEW (recipe to build app container)
├── docker-compose.yml  ← NEW (orchestrates all 3 containers)
├── .dockerignore       ← NEW (what to exclude from container)
├── .env.docker         ← NEW (environment variables for Docker)
└── init_db.py          ← NEW (auto-creates database tables)
```

STEP-BY-STEP IMPLEMENTATION

STEP 1: Create `backend/.dockerignore`

What is this? Just like `.gitignore` tells Git what to ignore, `.dockerignore` tells Docker what NOT to copy into the container. We don't want `venv`, cache files, or local environment files inside our container.

Where to create: `backend/.dockerignore` (same level as `requirements.txt`)

Create the file and paste this content:

```
# Virtual Environment (container installs its own packages)
```

```
venv/
```

```
env/
```

```
ENV/
```

```
# Python cache files
```

```
__pycache__/
```

```
*.py[cod]
```

```
*$py.class
```

```
*.so
```

```
# Environment files (we use .env.docker separately)
```

```
.env
```

```
.env.local
```

```
.env.docker
```

```
# Data files (container starts fresh)
```

```
data/faiss_index/
```

```
data/documents/*.pdf
```

```
data/metadata.json
```

```
# IDE files
```

```
.vscode/
```

```
.idea/
```

```
# OS files
```

```
.DS_Store
```

```
Thumbs.db
```

```
# Git
```

```
.git/
```

```
.gitignore
```

```
# Testing
```

```
.pytest_cache/
```

```
.coverage
```

```
htmlcov/
```

STEP 2: Create Dockerfile

What is this? The recipe that tells Docker how to build your FastAPI app into a container. It reads top to bottom like instructions:

1. Start with Python 3.11
2. Install system packages needed for PostgreSQL driver and FAISS
3. Install Python packages from requirements.txt
4. Copy your code

5. Start the server

Where to create: backend/Dockerfile (same level as requirements.txt)

Create the file and paste this content:

dockerfile

```
# =====
# Stage 1: Start with Python 3.11 base image
# Think of this as: "Give me a mini Linux with Python installed"
# =====
FROM python:3.11-slim

# =====
# Set working directory inside the container
# All commands after this run inside /app
# =====
WORKDIR /app

# =====
# Install system-level dependencies
# - build-essential: needed to compile some Python packages
# - libpq-dev: needed for psycopg2 (PostgreSQL driver)
# =====
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# =====
# Copy requirements.txt FIRST (Docker caching trick!)
# If requirements don't change, Docker reuses cached layer
# This saves 2-3 minutes on rebuilds
# =====
COPY requirements.txt .

RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt
```



```

# =====
# NOW copy the rest of your code
# This layer rebuilds when code changes
# But packages layer above stays cached!
# =====
COPY . .

# =====
# Create data directories the app needs
# =====
RUN mkdir -p data/faiss_index data/documents

# =====
# Tell Docker this container listens on port 8000
# =====
EXPOSE 8000

# =====
# Start command:
# 1. First run init_db.py to create/verify tables
# 2. Then start the FastAPI server
# =====
CMD ["sh", "-c", "python init_db.py && uvicorn app.main:app --host 0.0.0.0 --port 8000"]

```

Why python init_db.py && uvicorn ...? Docker creates a FRESH PostgreSQL database with no tables. The init_db.py script automatically creates your documents table, indexes, and triggers before the server starts. The && means "only start the server if table creation succeeded."

STEP 3: Create .env.docker

What is this? Environment variables specifically for the Docker setup. The key difference from your local .env is the hostnames:

- Local: localhost (talks to services on your machine)

- Docker: postgres and redis (talks to services by container name)

Where to create: backend/.env.docker (same level as requirements.txt)

Create the file and paste this content:

env

```
# =====
# LLM Configuration
# =====
GEMINI_API_KEY=your_gemini_api_key_here

# =====
# Embeddings
# =====
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2

# =====
# Vector Store
# =====
FAISS_INDEX_PATH=data/faiss_index/

# =====
# Metadata
# =====
METADATA_PATH=data/metadata.json

# =====
# PostgreSQL Configuration
# IMPORTANT: "postgres" is the SERVICE NAME from docker-compose.yml
# NOT "localhost" — containers talk to each other by service name!
# =====
DATABASE_URL=postgresql://raguser:ragpassword123@postgres:5432/ragdb
DB_POOL_SIZE=10
DB_MAX_OVERFLOW=20

# =====
# Redis Configuration
```

IMPORTANT: "redis" is the SERVICE NAME from docker-compose.yml

NOT "localhost" — same reason as above!

=====

REDIS_HOST=redis

REDIS_PORT=6379

REDIS_DB=0

REDIS_PASSWORD=

CACHE_TTL=3600

ENABLE_CACHING=true

=====

Server

=====

HOST=0.0.0.0

PORT=8000

⚠ IMPORTANT: Replace `your\_gemini\_api\_key\_here` with your actual Gemini API key!

****Why different hostnames?****

...

Your Mac (local development):

App → localhost:5432 → PostgreSQL on your Mac

App → localhost:6379 → Redis on your Mac

Docker (containerized):

App Container → postgres:5432 → PostgreSQL Container

App Container → redis:6379 → Redis Container

Inside Docker, containers find each other by SERVICE NAME,
not by localhost. "postgres" and "redis" are the names
we define in docker-compose.yml.

STEP 4: Create docker-compose.yml

What is this? The orchestration file that defines ALL services (containers) your project needs and how they connect. Think of it as the "full meal plan" — it says "start these 3 containers together, connect them, and make sure they're healthy."

Where to create: backend/docker-compose.yml (same level as requirements.txt)

Create the file and paste this content:

yaml

```
services:
  # =====
  # SERVICE 1: YOUR FASTAPI APPLICATION
  # =====
  app:
    # Build from Dockerfile in current directory
    build:
      context: .
      dockerfile: Dockerfile

    # Give the container a readable name
    container_name: rag-backend

    # Map port: YOUR_MACHINE:CONTAINER
    # Access your API at http://localhost:8000
    ports:
      - "8000:8000"

    # Load environment variables from .env.docker
    env_file:
      - .env.docker

    # IMPORTANT: Don't start the app until PostgreSQL and Redis are healthy!
    # This prevents "connection refused" errors on startup
    depends_on:

      postgres:
        condition: service_healthy
```

```
redis:
  condition: service_healthy
```

```
# Persist FAISS index data between container restarts
```

```
volumes:
```

```
- faiss_data:/app/data/faiss_index
```

```
# Auto-restart if the container crashes
```

```
restart: unless-stopped
```

```
# =====
```

```
# SERVICE 2: POSTGRES SQL DATABASE
```

```
# =====
```

```
postgres:
```

```
# Use official PostgreSQL 16 Alpine image (lightweight)
```

```
image: postgres:16-alpine
```

```
container_name: rag-postgres
```

```
# These create the database and user automatically on first start
environment:
```

```
  POSTGRES_USER: raguser
```

```
  POSTGRES_PASSWORD: ragpassword123
```

```
  POSTGRES_DB: ragdb
```

```
# Map port 5433 on your Mac to 5432 in container
```

```
# Why 5433? To avoid conflict if you have local PostgreSQL on 5432
```

```
ports:
```

```
- "5433:5432"
```

```
# Persist database data between container restarts
```

```
# Without this, you'd lose all data every time you restart!
```

```
volumes:
```

```
- postgres_data:/var/lib/postgresql/data
```

```
# Health check: Docker pings PostgreSQL every 5 seconds
```

```
# App container won't start until this passes
```

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U raguser -d ragdb"]
  interval: 5s
  timeout: 5s
  retries: 5
```

```
restart: unless-stopped
```

```
# =====
```

```
# SERVICE 3: REDIS CACHE
```

```
# =====
```

```
redis:
```

```
  # Use official Redis 7 Alpine image (lightweight)
```

```
  image: redis:7-alpine
```

```
  container_name: rag-redis
```

```
  # Map port 6380 on your Mac to 6379 in container
```

```
  # Why 6380? To avoid conflict if you have local Redis on 6379
```

```
  ports:
```

```
    - "6380:6379"
```

```
  # Persist Redis data
```

```
  volumes:
```

```
    - redis_data:/data
```

```
  # Health check: Docker pings Redis every 5 seconds
```

```
  healthcheck:
```

```
    test: ["CMD", "redis-cli", "ping"]
```

```
    interval: 5s
```

```
    timeout: 5s
```

```
    retries: 5
```

```
restart: unless-stopped
```

```
# =====
```

```
# VOLUMES: Persistent storage
```

Data in these volumes survives container restarts
Think of volumes as "external hard drives" for containers

=====

volumes:

postgres_data: # Stores PostgreSQL database files
redis_data: # Stores Redis cache data
faiss_data: # Stores FAISS vector index

****Understanding the Port Mapping:**

ports:

- "5433:5432"

├──┐
├──┐ Port INSIDE the container (PostgreSQL default)
├──┐
└──┐ Port on YOUR MACHINE (we use 5433 to avoid conflicts)

So:

From your app container → connects to postgres:5432 (internal)
From your Mac terminal → connects to localhost:5433 (external)
` ``

****Understanding Health Checks:****

` ``

Without health checks:

1. PostgreSQL container starts (takes 3 seconds to be ready)
2. App container starts IMMEDIATELY
3. App tries to connect to PostgreSQL → FAILS (not ready yet!)
4. App crashes ❌

With health checks:

1. PostgreSQL container starts
2. Docker pings PostgreSQL every 5 seconds: "Are you ready?"
3. After PostgreSQL says "yes" → THEN app container starts
4. App connects to PostgreSQL → SUCCESS ✅

STEP 5: Create init_db.py

What is this? When Docker creates a fresh PostgreSQL container, it has an empty database (no tables). This script automatically creates your documents table with all 13 columns, 6 indexes, and the full-text search trigger. It runs every time the container starts, but it's safe — it uses "IF NOT EXISTS" so it won't duplicate anything.

Where to create: backend/init_db.py (same level as requirements.txt)

Create the file and paste this content:

python

```
"""
Database initialization script.
Creates all tables, indexes, and triggers automatically.
Runs on every container startup (safe — uses IF NOT EXISTS).
"""
from sqlalchemy import text
from core.database import engine, Base, Document

def init_database():
    """Create all tables and setup full-text search"""
    print("🔧 Initializing database...")

    # 1. Create all tables from SQLAlchemy models
    # This reads your Document model and creates the table
    Base.metadata.create_all(bind=engine)
    print("✅ Tables created")

    # 2. Create additional indexes and triggers for full-text search
    with engine.connect() as conn:
        # Add tsvector column trigger (auto-updates content_tsv when content changes)
        conn.execute(text("""
            DO $$
            BEGIN
                -- Create trigger function if not exists
```



```

IF NOT EXISTS (
    SELECT 1 FROM pg_proc WHERE proname = 'documents_tsv_trigger'
) THEN
    CREATE FUNCTION documents_tsv_trigger() RETURNS trigger AS $func$
    BEGIN
        NEW.content_tsv := to_tsvector('english', COALESCE(NEW.content, ''));
        RETURN NEW;
    END
    $func$ LANGUAGE plpgsql;
END IF;

-- Create trigger if not exists
IF NOT EXISTS (
    SELECT 1 FROM pg_trigger WHERE tgname = 'tsvectorupdate'
) THEN
    CREATE TRIGGER tsvectorupdate
    BEFORE INSERT OR UPDATE ON documents
    FOR EACH ROW EXECUTE FUNCTION documents_tsv_trigger();
END IF;
END
$$;
"""))

# Create GIN index for full-text search if not exists
conn.execute(text("""
    CREATE INDEX IF NOT EXISTS idx_documents_content_tsv
    ON documents USING GIN(content_tsv);
"""))

# Create other indexes
conn.execute(text("""
    CREATE INDEX IF NOT EXISTS idx_documents_dates
    ON documents(valid_from, valid_to);
"""))

conn.execute(text("""
    CREATE INDEX IF NOT EXISTS idx_documents_source

```

```
        ON documents(source);
    """))

conn.execute(text("""
    CREATE INDEX IF NOT EXISTS idx_documents_version
    ON documents(version);
    """))

conn.execute(text("""
    CREATE INDEX IF NOT EXISTS idx_documents_latest
    ON documents(is_latest);
    """))

conn.commit()
print("✅ Indexes and triggers created")

print("🎉 Database initialization complete!")

if __name__ == "__main__":
    init_database()
```

RUNNING YOUR DOCKERIZED PROJECT

First Time Setup

Step 1: Make sure Docker Desktop is open and running (whale icon  in menu bar)

Step 2: Open Terminal and navigate to your backend folder:

```
bash
```

```
cd path/to/your/time-aware-rag/backend
```

Step 3: Build and start everything:

```
bash
```

```
docker-compose up --build
```

What happens behind the scenes:

Step 1: Docker downloads PostgreSQL image (~80MB, first time only)

Step 2: Docker downloads Redis image (~30MB, first time only)

Step 3: Docker builds your app image:

- Starts with Python 3.11
- Installs system packages
- Installs pip packages from requirements.txt
- Copies your code

Step 4: Docker creates a network so containers can talk

Step 5: Starts PostgreSQL container → waits for health check

Step 6: Starts Redis container → waits for health check

Step 7: Both healthy → starts your app container

Step 8: App runs init_db.py → creates tables

Step 9: App starts uvicorn → API is live!

First build takes 3-5 minutes. Subsequent builds are much faster (30 seconds) because Docker caches the package installation layer.

Step 4: You'll see logs scrolling. Wait until you see:

rag-backend |  Initializing database...

rag-backend |  Tables created

rag-backend |  Indexes and triggers created

rag-backend |  Database initialization complete!

rag-backend | INFO: Uvicorn running on <http://0.0.0.0:8000>

Step 5: Open your browser and go to:

<http://localhost:8000/docs>

You should see your FastAPI Swagger UI with all your endpoints!

ESSENTIAL DOCKER COMMANDS

Daily Commands (Use These Every Day)

bash

```
# Start all containers
docker-compose up

# Start in background (no log spam in terminal)
docker-compose up -d

# Stop all containers
docker-compose down

# Rebuild after ANY code change
docker-compose up --build

# Rebuild in background
docker-compose up --build -d

# See what's running
docker ps

# See logs (when running in background)
docker-compose logs -f

# See logs for specific container only
docker-compose logs -f app
docker-compose logs -f postgres
docker-compose logs -f redis
```

Debugging Commands (When Something Goes Wrong)

bash

```
# Go INSIDE your app container (like SSH into a server)
docker exec -it rag-backend bash

# Now you're inside! Try: ls, python --version, pip list
```

```
# Type 'exit' to come back out

# Go inside PostgreSQL and run SQL queries
docker exec -it rag-postgres psql -U raguser -d ragdb
# Now you're in psql! Try: \dt (show tables), \d documents (show columns)

# Type '\q' to come back out

# Go inside Redis and test it
docker exec -it rag-redis redis-cli
# Now you're in redis! Try: ping (should reply PONG), keys * (show all keys)
# Type 'exit' to come back out
```

Cleanup Commands

bash

```
# Stop containers AND delete all data (fresh start)
docker-compose down -v
# The -v flag removes volumes (database data, redis data, faiss index)
# Use this when you want a completely fresh database

# Remove unused Docker images (free up disk space)
docker system prune

# Nuclear option: remove EVERYTHING Docker-related
docker system prune -a
```

COMMON SCENARIOS

Scenario 1: You Changed Your Python Code

bash

```
docker-compose up --build
```

The `--build` flag tells Docker to rebuild your app container with the new code.

Scenario 2: You Want a Fresh Database

bash

```
docker-compose down -v
docker-compose up --build
```

The -v flag deletes all volumes (database data). On next start, init_db.py creates fresh tables.

Scenario 3: Something Crashed

bash

```
# Check what went wrong
docker-compose logs -f

# Restart everything
docker-compose down
docker-compose up --build
```

Scenario 4: Port 8000 Already in Use

Either stop whatever is using port 8000, or change the port in docker-compose.yml:

yaml

```
ports:
  - "8001:8000" # Now access your API at localhost:8001
```

Scenario 5: You Want to Connect to Docker's PostgreSQL from Your Local Machine

bash

```
psql -h localhost -p 5433 -U raguser -d ragdb
# Password: ragpassword123
# Note: port is 5433, not 5432!
```

📌 UNDERSTANDING THE ARCHITECTURE



📌 INTERVIEW TALKING POINTS

When an interviewer asks about Docker in your project, here's what you say:

> "I containerized the entire backend using Docker. The project runs as a multi-container setup with docker-compose — the FastAPI application, PostgreSQL database, and Redis cache each run in isolated containers. I configured health checks so the app container waits for PostgreSQL and Redis to be healthy before starting. The database schema including tables, indexes, and full-text search triggers is auto-initialized on startup. I used Docker volumes for data persistence so nothing is lost on container restarts. The entire stack can be started with a single docker-compose up command on any machine."

Key technical terms you used:

- Containerized
- Multi-container setup
- docker-compose orchestration
- Health checks
- Service dependency (depends_on with conditions)
- Data persistence (named volumes)
- Auto-initialization
- Isolated containers
- Port mapping

📌 QUICK REFERENCE CARD

FILE	PURPOSE	LOCATION
.dockerignore	What NOT to copy into container	backend/
Dockerfile	Recipe to build app container	backend/
docker-compose.yml	Orchestrate all 3 containers	backend/
.env.docker	Environment vars for Docker	backend/
init_db.py	Auto-create tables on startup	backend/

COMMAND	WHAT IT DOES
<code>docker-compose up --build</code>	Build and start everything
<code>docker-compose up -d</code>	Start in background
<code>docker-compose down</code>	Stop everything
<code>docker-compose down -v</code>	Stop and delete all data
<code>docker-compose logs -f</code>	Watch live logs
<code>docker ps</code>	See running containers
<code>docker exec -it <name> bash</code>	Go inside a container

📌 CHECKLIST

Before running `docker-compose up --build`, verify:

```
[ ] Docker Desktop is open and running (whale icon visible)
[ ] You're in the backend/ directory in terminal
[ ] .env.docker has your real GEMINI_API_KEY
[ ] All 5 files are created in the correct locations
[ ] Dockerfile exists at backend/Dockerfile
[ ] docker-compose.yml exists at backend/docker-compose.yml
[ ] .dockerignore exists at backend/.dockerignore
[ ] .env.docker exists at backend/.env.docker
[ ] init_db.py exists at backend/init_db.py
```

After successful startup, verify:

```
[ ] Terminal shows "Uvicorn running on http://0.0.0.0:8000"
[ ] http://localhost:8000 returns health check
[ ] http://localhost:8000/docs shows Swagger UI
[ ] docker ps shows 3 containers running
```